These types of services should be lightweight, stateless services for which the cost of instantiation is relatively low. They are destroyed when the service scope is destroyed. In the context of an ASP.NET Core application, the scope is destroyed at the end of an HTTP request. If you have a complex dependency graph where the same service type is injected into multiple constructors, each consumer will receive its own instance of the service. The `SimpleCityService` is a good candidate for the transient lifetime, as it meets the definition of a service that maintains no state and has a low instantiation cost.

To see this working, you will inject a second instance of the service into the `PageModel` and render its value to the browser. Make the following changes to the `LifetimeDemoModel` class.

Listing 7.9 Injecting a second service to the `PageModel`

```
private readonly LifetimeDemoService _lifetimeDemoService;
private readonly LifetimeDemoService _secondService;                ❶
public LifetimeDemoModel(LifetimeDemoService lifetimeDemoService,
    LifetimeDemoService secondService)                               ❷
{
    _lifetimeDemoService = lifetimeDemoService;
    _secondService = secondService;                                  ❸
}

public Guid Value { get; set; }
public Guid SecondValue { get; set; }                                ❹
public void OnGet()
{
    Value = _lifetimeDemoService.Value;
    SecondValue = _secondService.Value;                              ❺
}
```

❶ Add a private field for the second service.

❷ Inject a second instance of the LifetimeDemoService.

❸ Assign it to the private field.

❹ Add another public property to the PageModel.

❺ Set its value to the second service's Value.